Project X V1

Four-person engineering handoff - post-quantum, end-to-end private USDC payments

Purpose

This is the V1 execution contract for Aditya, Swarnim, Manan, and Manya. It is written for direct handoff to coding agents: each section begins with the interface another module can consume, then lists primitives, build steps, tests, and handoff artifacts.

V1 product statement

A post-quantum private USDC payment system that breaks the depositor-to-spend link and prevents wallet-specific RPC or network metadata from linking the payer to the transaction.

Scope and truthfulness

V1 hides ring-member / depositor-to-spend linkage, pre-execution payment parameters inside Chainlink CRE, and client-IP association for sensitive payment, Graph, and wallet RPC traffic. It does not hide public deposits, fixed denomination, final recipient address, or settlement time. Never say 'leaks no data'.

V1 cuts

- No browser extension, mobile wallet, PQ stealth recipient, arbitrary amounts/change notes, swaps, Gateway transfer, or public intent scheduler.
- No production TEE-attestation claim unless an attested deployment actually ships.
- No ECDH, ECDSA, EdDSA, or elliptic-curve primitive in wallet authority, ring, or relay encryption. A dev EOA may deploy/fund only; it is not payment authority.

1. Ownership and boundaries

Owner	Owns	Does not own
Aditya	Ring-payment protocol and contracts; Graph indexing/schemas and both selection algorithms.	Relay operations, CRE code, frontend pages, wallet secret storage.
Manya + Swarnim	Three-hop relay services, CRE confidential workflow, and typed frontend integration adapters. Manya owns the frontend integration experience; Swarnim owns the adapter implementation.	Ring proof/circuit and Graph selection policy; neither duplicates the other's boundary.
Manan	Browser-native PQ wallet SDK, registry/validator contracts, wallet key lifecycle.	Pool/ring contracts, node infrastructure, product pages.
Manya	All UX/pages, local-note UX, error display, demo polish; consumes adapters.	Crypto primitives, contract logic, node/CRE servers.

Integration rule

Each module may depend only on Section 2 exports. Internal key formats, storage, queue implementation, Graph queries, and contract internals are private. Breaking an export requires a version bump, a fixture update, and notice to every consumer before merge.

Suggested ownership paths

Area	Path	Owner
Wallet	packages/pq-wallet/; contracts/src/wallet/	Manan
Ring/pool	packages/ring-client/; contracts/src/ring/	Aditya
Graph	graph/	Aditya
Mesh/CRE	backend/mesh/; backend/cre/	Swarnim
Adapters	frontend/src/lib/protocol/	Manya + Swarnim
Product pages	frontend/src/	Manya
ABI/types	packages/protocol-types/	contract owner publishes

Every-owner definition of done

- Public API plus a runnable happy-path and failure example.
- Tests for valid, malformed, replayed, and unavailable-dependency behavior.
- Mock with exactly the same public shape so downstream owners progress independently.
- README: setup command, environment variables, expected output, known limitation.

2. Frozen end interfaces

Shared protocol types

```
type Hex = string; // 0x-prefixed lower-case bytes
type NoteCommitment = Hex; type Nullifier = Hex; type TxHash = Hex;
type Denomination = 1_000_000 | 5_000_000 | 10_000_000; // USDC base units
interface LocalNote { secret: Hex; commitment: NoteCommitment; denomination: Denomination; createdAtBlock: bigint; pool: Hex; }
interface PaymentRequest { recipient: Hex; denomination: Denomination; note: LocalNote; }
interface PrivateSpend { ring: NoteCommitment[]; nullifier: Nullifier; proof: Hex; recipient: Hex; denomination: Denomination; paym
```

Non-negotiable nullifier rule

nullifier = H(NULLIFIER_DOMAIN, noteSecret, poolId). It never contains recipient or paymentContext. Otherwise the same note can be spent twice by varying recipient. paymentContext = H(PAYMENT_DOMAIN, poolId, chainId, recipient, denomination) is separately bound inside the proof.

Manan exports: wallet

```
interface PqWallet {
  create(): Promise<{accountAddress: Hex; pkCommitment: Hex}>;
  register(): Promise<TxHash>;
  signAction(payload: Hex): Promise<{digest: Hex; signature: Hex; useCount: bigint}>;
  getState(): Promise<{accountAddress: Hex; pkCommitment: Hex; useCount: bigint; maxUses: bigint; rotationDeadline: bigint; active:
  rotate(nextKey: Uint8Array): Promise<TxHash>;
}
```

Aditya exports: settlement and Graph

```
interface RingClient {
  createNote(denomination: Denomination, pool: Hex): Promise<LocalNote>;
  getSpendableNotes(pool: Hex): Promise<LocalNote[]>;
  buildSpend(request: PaymentRequest, decoys: NoteCommitment[]): Promise<PrivateSpend>;
  verifyLocally(spend: PrivateSpend): Promise<boolean>;
}
interface PrivatePoolContract {
  deposit(commitment: NoteCommitment, denomination: Denomination): Promise<TxHash>;
  spend(spend: PrivateSpend): Promise<TxHash>;
  isNullifierSpent(nullifier: Nullifier): Promise<boolean>;
}
interface RelayNode { id:string; endpoint:string; kemPublicKey:Hex; reliabilityScore:bigint; batchOccupancy:bigint; recentSelection
interface GraphSelectionClient {
  selectRing(real: NoteCommitment, denomination: Denomination): Promise<{ring:NoteCommitment[]; reasons:string[]; poolSize:number;
  selectPath(): Promise<{nodes:[RelayNode,RelayNode,RelayNode]; probabilities:number[]}>;
}
```

Swarnim exports: mesh, CRE, adapter barrel

```
interface PrivacyTransport {
  submitPayment(spend: PrivateSpend, path: RelayNode[]): Promise<{messageId:string}>;
  query<T>(request: MeshQuery, path: RelayNode[]): Promise<T>;
  subscribe(messageId:string, listener:(event:MeshStatusEvent)=>void):()=>void;
}
interface ConfidentialPolicyClient {
  authorize(input:{spendHash:Hex; recipient:Hex; denomination:Denomination}):
    Promise<{authorization:Hex; expiresAt:bigint; auditId:string}>;
}
// Manya imports only frontend/src/lib/protocol/index.ts. No raw GraphQL/RPC/ABI/relay calls in pages.
```

3. Manan - PQ wallet

Deliverable

An app-native, browser-local PQ smart-wallet authority module. It creates a smart-account identity, commits to a hash-based public key, and authorizes actions via the PQ validator. V1 is a web wallet, not an extension.

Primitives and architecture

- Generate secrets locally with WebCrypto CSPRNG. Persist V1 state in IndexedDB; encrypt at rest where feasible. Never transmit raw seed, secret key, browser storage key, or signer tree state.
- Use the chosen FORS+C-style few-time hash-based signer. Make k, a, maxUses, signature size, and benchmark result explicit. Do not claim an unmeasured gas/security number.
- PQKeyRegistry holds pkCommitment, nextCommitment, useCount, maxUses, rotationDeadline, disableAfter. PQValidator calls canonical digest + verifier.
- Use Arc ERC-4337 support for smart-account plumbing. A bundler/paymaster is transport/gas infrastructure only; PQValidator is the authority.

Canonical digest

```
digest = keccak256(abi.encode(
  PQ_DOMAIN, chainId, walletAddress, schemeId, useCount, keccak256(payload)
));
// payload = ABI(target, value, calldataHash, expiry)
// Expose digestFor() on-chain. Frontend never reproduces ABI hashing itself.
```

Contracts/functions

```
register(pkCommitment, nextCommitment, maxUses, rotationDeadline)
rotate(nextCommitment, currentPqSignature)
disable(currentPqSignature) // starts timelock
getState(account) -> PQKeyState
digestFor(account, payload, schemeId, useCount) -> bytes32
validateUserOp(userOp, userOpHash) -> validationData
```

Build sequence

- Publish SDK type shell and deterministic known-answer test vectors before UI.
- Implement pure keygen/sign/commitment functions, with same vectors in TypeScript and Solidity tests.
- Implement registry tests: registration, exhaustion, PQ-authenticated rotation, expired rotation, disable timelock, cross-account isolation.
- Implement validator/digest parity. Publish a conforming mock PqWallet immediately.
- Handoff: deployed addresses, ABI/types, package, vectors, and one create/register script.

Acceptance tests

Test	Expected
Replay at old useCount	Rejected.
Mutate chain/wallet/scheme/payload	Digest changes; rejection.
useCount=maxUses	Next action rejects.
ECDSA-only rotation	No valid route / reject.
Browser refresh	Local state restores and matches registry.
Secret leakage scan	No secret in adapter, Graph, CRE, mesh, or logs.

4. Aditya - ring payments and Graph

Deliverable

The confidential payment core: fixed-denomination pooled notes, 1-of-8 note opening, proof verifier adapter/fallback, nullifier safety, Arc USDC pool release, and Graph-driven decoy/hop selection.

Required primitives

- noteSecret: 32 local random bytes. commitment = H(NOTE_DOMAIN, noteSecret, poolId, denomination). A commitment is public; no address-to-balance mapping exists.
- nullifier = H(NULLIFIER_DOMAIN, noteSecret, poolId). This single invariant stops double spending.
- paymentContext = H(PAYMENT_DOMAIN, poolId, chainId, recipient, denomination), supplied as public proof input.
- Ring is real note plus seven same-denomination decoys. Canonical-sort all eight commitments before proving and verifying.
- Use MPC-in-the-head/hash-symmetric proof only if feasibility spike passes. This is a symmetric/hash zero-knowledge proof; say so accurately.

Day-1 cryptography gate

Report 1-of-8 proof generation time, bytes, verification gas, and test transaction result. If it fails, use a PQ single-note signer fallback behind identical pool/spend interface. Do not block Graph, mesh, CRE, Arc, or frontend; label fallback in README/demo.

Pool contract

```
mapping(bytes32=>bool) public commitments;
mapping(bytes32=>bool) public spentNullifiers;
mapping(uint256=>bool) public allowedDenomination;
function deposit(bytes32 commitment,uint256 denomination) external;
function spend(bytes32[8] ring,bytes proof,bytes32 nullifier,address recipient,uint256 denomination) external;
event NoteDeposited(bytes32 indexed commitment,uint256 denomination,uint64 indexed epoch);
event PrivateSpendExecuted(bytes32 indexed nullifier,uint256 denomination,address indexed recipient,bytes32 ringHash);
```

spend() atomically verifies proof, validates ring and denomination, rejects/sets nullifier, and releases pool USDC. Never emit real signer slot, note secret, or per-payment decoy mapping.

Graph schemas and policies

```
type RingMember @entity {
 id:ID! denomination:BigInt! enrolledAt:BigInt! timesUsedInRing:Int!
 lastUsedAt:BigInt fundingSourceCluster:String! hasOtherActivity:Boolean!
}
type RingPool @entity { id:ID! poolSize:Int! freshnessScore:BigInt! }
type RelayNode @entity {
 id:ID! endpoint:String! kemPublicKey:Bytes! reliabilityScore:BigInt!
 batchOccupancy:BigInt! recentSelectionCount:Int! lastSeenAt:BigInt!
}
```

- selectRing: same denomination -> exclude real/overused -> penalize dominant funding cluster -> prefer organic activity -> sample enrollment ages -> real plus seven -> canonical sort.
- selectPath: reliability floor, then weight=batchOccupancy/(1+recentSelectionCount); use Markov draw for three distinct nodes, never deterministic top-three.
- Deploy/query live Graph data for submission. Fixture mode matches type shape but is labeled local/demo.

Tests and handoff

- Tests: duplicate commitment, unsupported denomination, 7/9/repeated ring, nullifier replay to different recipient, recipient tamper after proof, event privacy.
- Publish MockVerifier and RealRingVerifier same ABI; other owners never import proof library.
- Test Graph empty pool and unavailable provider with typed errors. Index no note secret or real-member mapping.

- Handoff deployed pool/verifier addresses, ABI/types, RingClient, GraphSelectionClient, live endpoint/config, fixture, deposit/spend script.

5. Swarnim - mesh, CRE, and adapters

Deliverable

A three-hop encrypted relay mesh, confidential Chainlink payment-policy gate, and frontend adapter layer. Mesh transports opaque envelopes; it never sees local note secrets or owns proof correctness.

Mesh design

- Run three independent relays. Validate envelope, queue fixed batch window, add bounded random delay, forward one hop. Final hop submits PAYMENT or forwards allowlisted QUERY.
- Use ML-KEM-768 per-hop encapsulation, HKDF-SHA-256 key derivation, AES-256-GCM onion encryption. No X25519/ECDH in product privacy path.
- Layer fields: version, message ID, expiry, hop ID, KEM ciphertext, AEAD nonce, encrypted inner layer. Reject malformed, expired, replayed messages.
- QUERY has one-time response key and return route. Final hop accepts only RPC/Graph allowlist, never arbitrary URLs. Encrypt result back through route.
- If TEE attestation is simulated, publish simulatedAttestation=true in metadata and label it plainly.

Relay API

```
type MeshMessage={version:1;messageId:string;type:"PAYMENT"|"QUERY";expiresAt:number;
 kemCiphertext:Hex;nonce:Hex;ciphertext:Hex;}
POST /v1/relay -> {accepted:true,messageId:string}
GET /v1/status/:id -> {state:"QUEUED"|"BATCHED"|"FORWARDED"|"SUBMITTED"|"FAILED";
 updatedAt:number;publicReason?:string}
```

Allowed logs: message ID, hop ID, queue timestamp, status. Forbidden logs: plaintext, source IP linked to payload, note secret, proof witness, recipient policy data, AEAD key.

Chainlink CRE

handlerInTee takes recipient, denomination, spendHash, credential/secret. It checks policy confidentially, then returns short-lived authorization bound to spendHash. Final payment submission requires it; this is a core state-changing workflow, not a decorative check.

```
handlerInTee(async ({recipient,denomination,spendHash,credential})=>{
 const ok=await confidentialComplianceCheck(recipient,credential);
 if(!ok)return {approved:false,reason:"POLICY_DENIED"};
 const expiresAt=now()+120;
 return {approved:true,authorization:H(CRE_DOMAIN,spendHash,expiresAt,workflowSecret),expiresAt};
});
```

Build/test/handoff

- Create local Docker 3-node topology and deterministic test keys; PAYMENT first, then QUERY return routing.
- Publish frontend/src/lib/protocol/index.ts: PqWalletAdapter, RingPaymentAdapter, GraphSelectionAdapter, PrivacyTransportAdapter, ConfidentialPolicyAdapter.
- Start from CRE Confidential Workflow starter, implement actual handlerInTee, run simulation, preserve logs.
- Tests: plaintext reject, expiry/replay, relay failure, non-allowlisted URL, query returns through route, policy denial, copied authorization on altered spendHash.
- Handoff compose/runbook, node endpoints/KEM public keys, CRE source/evidence, adapter barrel, mock fixture.

6. Manya - product frontend

Deliverable

A clean judge flow that explains the protocol precisely and never puts sensitive data in UI telemetry or logs. Manya owns pages and state, consuming only Swarnim's adapter barrel.

Screen	Action	Consumes
Welcome/disclosure	Read protected vs public fields.	Static claim copy.
Create wallet	Create/register PQ smart account.	PqWalletAdapter.
Deposit	Choose fixed denomination; create note/deposit.	RingPaymentAdapter + wallet.
Private payment	Public recipient; choose note; begin flow.	Graph -> Ring -> CRE -> Mesh adapters.
Privacy route	Show 8-note and 3-hop safe status.	Selection reasons/status only.
Receipt/failure	Arc result or typed failure.	Pool event/mesh status.

UI state machine

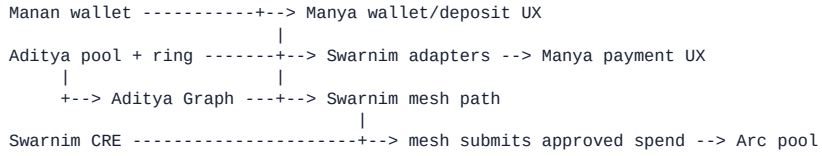
```
WALLET_READY -> NOTE_SELECTED -> GRAPH_SELECTING -> RING_FORMED
-> POLICY_CHECKING -> MESH_QUEUED -> MESH_BATCHED -> RELAYED
-> SETTLEMENT_PENDING -> SETTLED | FAILED
Errors: INSUFFICIENT_ANONYMITY, GRAPH_UNAVAILABLE, POLICY_DENIED,
POLICY_UNAVAILABLE, MESH_UNAVAILABLE, PROOF_REJECTED, NULLIFIER_SPENT,
UNSUPPORTED_DENOMINATION, SETTLEMENT_REVERTED.
```

Rules/tests/handoff

- No raw GraphQL/RPC/ABI/relay calls in pages. Adapter gap -> report, never bypass.
- Warn users cleared browser storage loses V1 notes; no recovery promise.
- No note secrets, wallet secrets, proof witness, CRE credentials, raw query, or KEM material in React state, URL, telemetry, console, or errors.
- Show '8 eligible notes selected' / '3 relays selected', never signer slot or source IP.
- Tests: fresh user completes wallet/deposit; insufficient anonymity blocks; relay status advances; policy denial leaks nothing; refresh restores status only; disclosure says recipient/denomination/time are public.
- Handoff final Arc run, video-ready screens, demo script, screenshots, limitation copy matching README.

7. Integration gates and execution

Dependency graph



Gate	Owner	Pass condition	Failure response
G0 shared types	All	Section 2 types compile; mocks run.	Fix API mismatch first.
G1 proof	Aditya	1-of-8 metrics + test transaction.	Single-note PQ fallback.
G2 wallet	Manan	Replay/mutation reject live.	Mock integration only; no final claim.
G3 QUERY privacy	Swarnim	Browser receives Graph/RPC via return route.	Do not claim RPC privacy.
G4 CRE	Swarnim	handlerInTee gives spend-bound auth.	No Chainlink readiness claim.
G5 full run	Manya	One run finishes Arc settlement.	Cut polish; repair sequence.

Daily protocol

- Start-of-day: endpoint/type/address, gate state, one blocker.
- Handoff: command another owner runs, expected output, one failure behavior.
- Breaking API: version, fixture update, notify consumers before merge.
- Merge requires shared typecheck, contracts test, mesh integration test, frontend smoke run.

8. End-to-end verification and evidence

Case	Steps	Evidence
Happy path	Wallet -> deposit -> Graph -> CRE -> mesh -> Arc.	Arc tx; live Graph response; CRE log; mesh states; recording.
Double spend	One note, two recipients.	Second call rejects spent nullifier.
Tamper	Change recipient after proof/auth.	paymentContext/spendHash binding rejects.
Policy denial	Fail test credential.	No final submission.
RPC privacy	Fetch ring/node data.	QUERY return; no direct browser request.
Graph causal role	Alter metrics/eligibility.	Selected ring/path changes.

Sponsor submission evidence

Sponsor	Must capture
The Graph	Live provider query, deployed schema/indexer, proof data changes ring/path choice, public README.
Chainlink	handlerInTee source + successful sim/deploy, sensitive input in TEE, authorization changes execution.
Arc	Functional frontend/backend, architecture diagram, Arc Testnet USDC settlement, clear Arc/App Kit explanation.

README/demo language

- 'Post-quantum authority path' only after validator is live; never imply Arc L1 is PQ.
- 'Ring size eight, proof-of-concept'; do not overstate anonymity.
- Protects partial relay observation, not global passive adversary or all-hop collusion.
- Deposit, denomination, recipient, and time public in V1.
- Browser-local notes unrecoverable after storage loss.

Final V1 rule

Record one full end-to-end run before submission work. If any gate is incomplete, the README and video must say exactly which property is not shipped. New scope requires a versioned V1.1 revision, never an unannounced cross-module change.